Vibrational Density-of-States Normalization Specification

VS3: Explicit VDOS Normalization of a Velocity Spectrum

mdstats

2026-07-22

Contents

Purpose and implementation stage

This document specifies stage VS3 in

```
mdstats/analysis/velocity_spectrum.py
```

The public function is

```
compute_vdos(spectrum, ...)
```

It converts an existing `VelocitySpectrumResult` into an explicitly normalized finite-temperature vibrational density of states. It does not recompute a VACF, Fourier transform a trajectory, infer phonon eigenmodes, calculate infrared or Raman intensities, or apply two-phase thermodynamics.

Terminology and interpretation

The result is called a **vibrational density of states** (VDOS). The more specific label **phonon density of states** is justified only for a crystalline or nearly harmonic solid where collective normal-mode language remains physically meaningful.

For liquids, molten salts, highly anharmonic solids, and diffusive ions, the same normalized function is a finite-temperature velocity-derived VDOS. Its low-frequency region may contain translation, diffusion, hopping, and cage motion in addition to oscillatory vibrations.

A VDOS is not an optical spectrum. Infrared intensity requires a dipole or charge-current correlation; Raman intensity requires a polarizability correlation.

Provenance

Borrowed physical background

Velocity-correlation spectra as descriptions of atomic motion are established in molecular dynamics, including Rahman's liquid-argon analysis [1]. The Wiener-Khinchin relation used by the source spectrum is attributed in the VS1 specification. Lin, Blanco, and Goddard later used a VACF-derived density of states in the two-phase thermodynamic method [2]. VS3 does **not** implement that method, its gas/solid decomposition, or any thermodynamic formula.

mdstats-specific contribution

The following are mdstats design decisions:

- separating the raw velocity spectrum from VDOS normalization;
- using the discrete one-sided FFT-bin measure rather than trapezoidal endpoint weights;
- requiring an explicit target for degrees-of-freedom normalization;
- refusing to infer removed constraints from atom count alone;
- applying one scalar factor to total, Cartesian, and per-atom projections;
- explicit low-frequency cropping without interpolation;
- strict material-negative rejection and roundoff-only clipping;
- the immutable result schema, validation identities, terminology metadata, and provenance payload.

Dependency boundary

VS3 consumes only

```
VelocitySpectrumResult
```

and the private helper

```
spectral_bin_integral
```

It does not depend on trajectory collections, velocity selection, VACF construction, or quadrature utilities.

Public function

```python
def compute_vdos(
    spectrum: VelocitySpectrumResult,
    *,
    normalization: Literal[
        "unit_area",
        "degrees_of_freedom",
        "none",
    ] = "unit_area",
    target_degrees_of_freedom: float | None = None,
    minimum_frequency_thz: float | None = None,
    negative_policy: Literal["error", "clip_roundoff"] = "clip_roundoff",
    negative_tolerance: float = 1.0e-12,
) -> VDOSResult:
    ...
```

Public result

```python
@dataclass(frozen=True, slots=True)
class VDOSResult:
    frequencies_thz: NDArray[np.float64]
```

```
    wavenumbers_cm_inv: NDArray[np.float64]
    energies_mev: NDArray[np.float64]

    total: NDArray[np.float64]                 # (F,)
    components: NDArray[np.float64]            # (F, 3)
    per_atom: NDArray[np.float64] | None       # (F, A)
    per_atom_components: NDArray[np.float64] | None  # (F, A, 3)
    per_atom_indices: NDArray[np.int64] | None

    normalization: str
    integrated_weight_before: float
    integrated_weight_after: float
    target_weight: float | None
    source_estimator: str
    weighting: str
    density_units: str
    metadata: Mapping[str, Any]
    signature: DynamicsInputSignature | None
```

Input contract

The input must satisfy the `VelocitySpectrumResult` constructor invariants and must record

```
spectral_sidedness == "one_sided"
spectral_scaling  == "density"
```

The frequency grid must contain at least two uniformly spaced bins. VS3 accepts both VACF-transform and later Welch source estimators.

Source weighting

Mass weighting is preferred when the result is interpreted as a vibrational DOS. Uniform or explicit-uniform weighting remains numerically valid, but the metadata records that the result is a velocity-derived VDOS rather than silently declaring it a harmonic phonon DOS.

Frequency selection

When `minimum_frequency_thz` is not `None`, VS3 retains existing bins satisfying

$$f_m \geq f_{\min}.$$

It does not interpolate a new boundary bin. The retained grid must contain at least two bins. The threshold and first retained source index are recorded in metadata.

This option can remove a DC or low-frequency diffusive contribution, but the operation changes the normalized object and must remain explicit.

Negative-value policy

Diagonal velocity spectra and VDOS projections should be nonnegative in the ideal infinite-sampling limit. A finite transformed VACF may contain negative lobes.

For each real projection, define a scale-aware threshold

$$\epsilon_P = \epsilon_{\text{user}} \max(1, \max|P|).$$

- `negative_policy="error"` rejects any negative sample.
- `negative_policy="clip_roundoff"` clips samples in $[-\epsilon_P, 0)$ and rejects values below $-\epsilon_P$.

Material negative weight is never silently discarded. After permitted clipping, total and per-atom scalar arrays are recomputed from their Cartesian components so exact trace identities are restored.

Normalization modes

`none`

No scalar area normalization is applied. The retained, validated spectrum is returned with its source density units.

`unit_area`

The target is

$$I_{\text{target}} = 1.$$

The result has density units `1/THz`.

`degrees_of_freedom`

The caller must supply a finite, strictly positive `target_degrees_of_freedom`. Examples include $3N$, $3N - 3$, or a user-defined projected count. VS3 does not guess translational, rotational, rigid-body, or constraint removals.

The density units are `degrees_of_freedom/THz`.

Supplying `target_degrees_of_freedom` for another normalization mode is an error rather than an ignored parameter.

Discrete normalization measure

For uniform one-sided FFT bins,

$$I = \Delta f \sum_m g_m.$$

VS3 calls `spectral_bin_integral`. It does not call `numpy.trapezoid`, SciPy quadrature, or an interpolating integrator. The one-sided endpoint weights were already handled by the spectral estimator.

Let I_0 be the retained pre-normalization weight and $I_\star$ the target. For normalized modes,

$$a = \frac{I_\star}{I_0}, \qquad g'_m = a g_m.$$

Exactly the same scalar a is applied to total, Cartesian, and per-atom projections.

Algorithm

```
1. Validate the result type and one-sided density metadata.
2. Validate the requested normalization and tolerance.
3. Resolve the existing-bin frequency slice.
4. Copy total, component, and per-atom projections on that slice.
5. Apply the explicit negative policy to component projections.
6. Recompute scalar projections from Cartesian traces.
7. Compute the retained total weight with spectral_bin_integral.
8. Reject nonpositive or nonfinite retained weight.
9. Resolve the target and one scalar normalization factor.
10. Scale every retained projection by the same factor.
11. Recompute the post-normalization bin measure.
12. Construct VDOSResult with source and operation provenance.
```

Result identities

The constructor validates

$$g(f) = g_x(f) + g_y(f) + g_z(f),$$

and, when per-atom components exist,

$$g_i(f) = g_{ix}(f) + g_{iy}(f) + g_{iz}(f).$$

For `unit_area`,

$$\Delta f \sum_m g_m = 1.$$

For degrees-of-freedom normalization,

$$\Delta f \sum_m g_m = N_{\mathrm{dof,target}}.$$

Edge cases and failure policy

VS3 rejects:

- non-`VelocitySpectrumResult` input;
- unsupported spectral sidedness or scaling;
- fewer than two retained frequency bins;
- invalid normalization or negative policy;
- a missing, nonfinite, or nonpositive degrees-of-freedom target;
- a target supplied for another normalization mode;
- a nonfinite or negative frequency threshold;
- nonfinite or negative tolerance;
- material negative total, Cartesian, or per-atom spectral values;
- zero or negative retained total weight.

No warning substitutes for an invalid normalization.

Complexity and memory

For F retained frequency bins and A stored per-atom projections, time and additional memory are

$$O(FA)$$

when per-atom data exist, and $O(F)$ otherwise. The source result is not modified.

Required tests

- unit-area normalization through the discrete bin measure;
- explicit $3N$, $3N-3$, and arbitrary target normalization;
- exact preservation for `normalization="none"` without cropping or clipping;
- total/component and per-atom/component trace identities;
- zero-padding invariance of normalized integrated weight;
- explicit low-frequency existing-bin cropping;
- rejection of a threshold that leaves fewer than two bins;
- material-negative rejection;
- roundoff-only clipping;
- no mutation of the source result;
- deliberate inequality between the normative bin sum and trapezoidal endpoint weighting;
- result-constructor shape, unit, and metadata validation;
- top-level public import smoke testing.

Deferred work

VS3 does not implement:

- direct Welch estimation;
- spectrum or VDOS plotting;
- peak finding or mode assignment;
- quantum correction factors;
- harmonic dynamical-matrix phonon DOS;
- 2PT entropy or free energy;

- uncertainty estimation.

H0 signature propagation and deep immutability

A `VDOSResult` preserves the input `VelocitySpectrumResult.signature`. VDOS normalization changes the spectral measure but not the trajectory, atom selection, drift, velocity source, or analysis subspace identity.

Every frequency/projection array is owned and read-only, and metadata is recursively immutable. A supplied signature must retain the full Cartesian source subspace. The common contract is `docs/specs/analysis/_dynamics_common_spec.md`.

References

[1] A. Rahman, "Correlations in the Motion of Atoms in Liquid Argon," *Physical Review* **136**, A405-A411 (1964). DOI: 10.1103/PhysRev.136.A405.

[2] S.-T. Lin, M. Blanco, and W. A. Goddard III, "The Two-Phase Model for Calculating Thermodynamic Properties of Liquids from Molecular Dynamics: Validation for the Phase Diagram of Lennard-Jones Fluids," *Journal of Chemical Physics* **119**, 11792-11805 (2003). DOI: 10.1063/1.1624057.